

GroAurum

Phase 2 Sprint 3 - Supabase Data Layer

Production-ready read architecture

Generated: July 16, 2026

No CRUD mutations - no UI redesign - mock until Supabase enabled

1. Sprint Goal

Replace typed fixtures as the page data source with a production-ready data access architecture: repository pattern, service layer, adapters, caching, shared query states, and TanStack Query. Pages load through repositories; fixtures remain mock seed only. Switching to Supabase requires changing one adapter (VITE_DATA_ADAPTER).

2. Architecture Overview

- packages/data - domain models, ReadRepository contract, query keys, query states, errors, realtime interfaces
- packages/api - MemoryCache, mock + Supabase stub adapters, AdminRepositories factory, AdminDataService
- apps/admin-web/src/data - typed client, mock sources, React Query hooks, QueryStateGate
- Pages call hooks -> repositories -> adapter; no direct fixture data loading on pages
- Read-only surface: list, getById, search, subscribe (noop realtime bus)

3. Layer Responsibilities

Layer	Responsibility
Domain (@groaurum/data)	Typed models, errors, QueryState, keys
Repository	ReadRepository<TList,TDetail> per entity
Adapter (@groaurum/api)	Mock seed vs Supabase stub implementations
Cache	MemoryCache TTL memoization at adapter
Service	AdminDataService facades + invalidation hooks
React Query	Network/UI cache, staleTime, query keys
UI gate	Loading / Empty / Error / Success

4. Folder Structure

```
packages/data/src/
  models.ts repository.ts query-keys.ts query-state.ts realtime.ts
packages/api/src/
  adapters/create-admin-repositories.ts
  repositories/create-read-repository.ts admin-repositories.ts
  cache/memory-cache.ts
  services/admin-data-service.ts
apps/admin-web/src/data/
  adminDataClient.ts buildAdminMockSources.ts
  AdminDataProviders.tsx hooks.ts QueryStateGate.tsx
  *-fixtures.ts (seed only) *-types.ts
```

5. Repositories

Each repository implements `list()`, `getById()`, `search()`, `subscribe()`. No create/update/delete in Sprint 3.

- products
- categories
- skus
- prices
- inventory
- customers
- orders
- salesmen
- delivery
- reports
- settings
- dashboard (console snapshot companion)

6. Domain Models

- Strict TypeScript contracts in `packages/data/src/models.ts` (no any)
- Admin ERP specializes `AdminRepositories` with view-model types (`ProductListRow`, `OrderDetail`, snapshots, etc.)
- Fixtures keep the same shapes; they are `EntitySource` seed inputs only
- Categories seeded in `buildAdminMockSources` for repository completeness

7. Caching Strategy

- Adapter MemoryCache: key prefixes entity:list | entity:detail:id | entity:search:q; default TTL 30s
- TanStack Query: staleTime 30s, gcTime 5m, retry 1, refetchOnWindowFocus false
- queryKeys factory: root, entity, list, detail, search, snapshot
- invalidation.entity(name) / invalidation.all() prepared for future CRUD
- AdminDataService.invalidate drops MemoryCache prefix and returns query key prefixes

8. Query Strategy

Page -> useXQuery hook

- > queryKey from queryKeys.*
- > queryFn: repositories.<entity>.list|getById|search
or client.getSnapshot for KPI+row consoles
- > toQueryState(...) -> QueryStateGate
loading | empty | error | success
- List pages: products, prices use repository.list
- Module consoles: inventory, customers, orders, salesmen, delivery use getSnapshot
- Detail pages: getById(id) with enabled when id present
- Reports / Settings / Dashboard: getById('current') via repository
- Pure UI helpers may still import fixture helpers (filters, browse builders)

9. Shared Query States

- QueryStatus: loading | empty | error | success
- DataError codes: not_found, unauthorized, forbidden, offline, timeout, adapter_unavailable, unexpected
- QueryStateGate renders PageHeader + EmptyState for non-success paths
- Success path renders children(data) without redesigning module UI

10. Supabase Integration Points

- Single switch: VITE_DATA_ADAPTER=mock|supabase (apps/admin-web/.env.example)
- createAdminRepositories({ mode }) builds mock repos or Supabase stubs
- createSupabaseReadRepositoryStub throws adapter_unavailable until wired
- Future: inject Supabase client; map tables to EntitySource / ReadRepository
- Keep queryKeys + hooks unchanged when swapping adapter implementations
- RealtimeBus + ReadRepository.subscribe ready; createNoopRealtimeBus today
- Do not open live channels in Sprint 3

11. Realtime Interfaces (prepared only)

RealtimeEventType / RealtimeEvent / RealtimeListener / Unsubscribe
 RealtimeBus.subscribe(entity, listener) -> Unsubscribe
 createNoopRealtimeBus() - no channels
 Mock repo subscribe listens for INVALIDATE and clears MemoryCache prefix

12. Future CRUD Hooks (not implemented)

- useCreateProductMutation / useUpdateProductMutation / useDeleteProductMutation
- useCreateOrderMutation / useUpdateInventoryMutation / useUpdateSettingsMutation
- On success: AdminDataService.invalidate(entity) + queryClient.invalidateQueries
- Mutation methods stay off ReadRepository until a dedicated write interface
- Exported type FutureMutationHookNames documents planned names

13. Validation

Check	Result
pnpm --filter @groaurum/data typecheck	Pass
pnpm --filter @groaurum/api typecheck	Pass
pnpm --filter @groaurum/api test	Pass
pnpm --filter @groaurum/admin-web typecheck	Pass
CRUD / mutations	Not implemented (by design)
Live Supabase reads	Stub only (by design)
UI redesign / business logic change	None
Realtime subscriptions	Interfaces only

14. How to Review

```
pnpm install
pnpm dev:admin
# Default VITE_DATA_ADAPTER=mock - pages load via repositories
# Confirm Loading gate flashes then module content renders
# Set VITE_DATA_ADAPTER=supabase - adapter_unavailable error states
# Fixtures only referenced from buildAdminMockSources (+ pure helpers)
```

15. Intent

Sprint 3 establishes a production-ready read data layer so Supabase and future CRUD can plug into stable contracts without rewriting admin modules or inventing per-page data access.